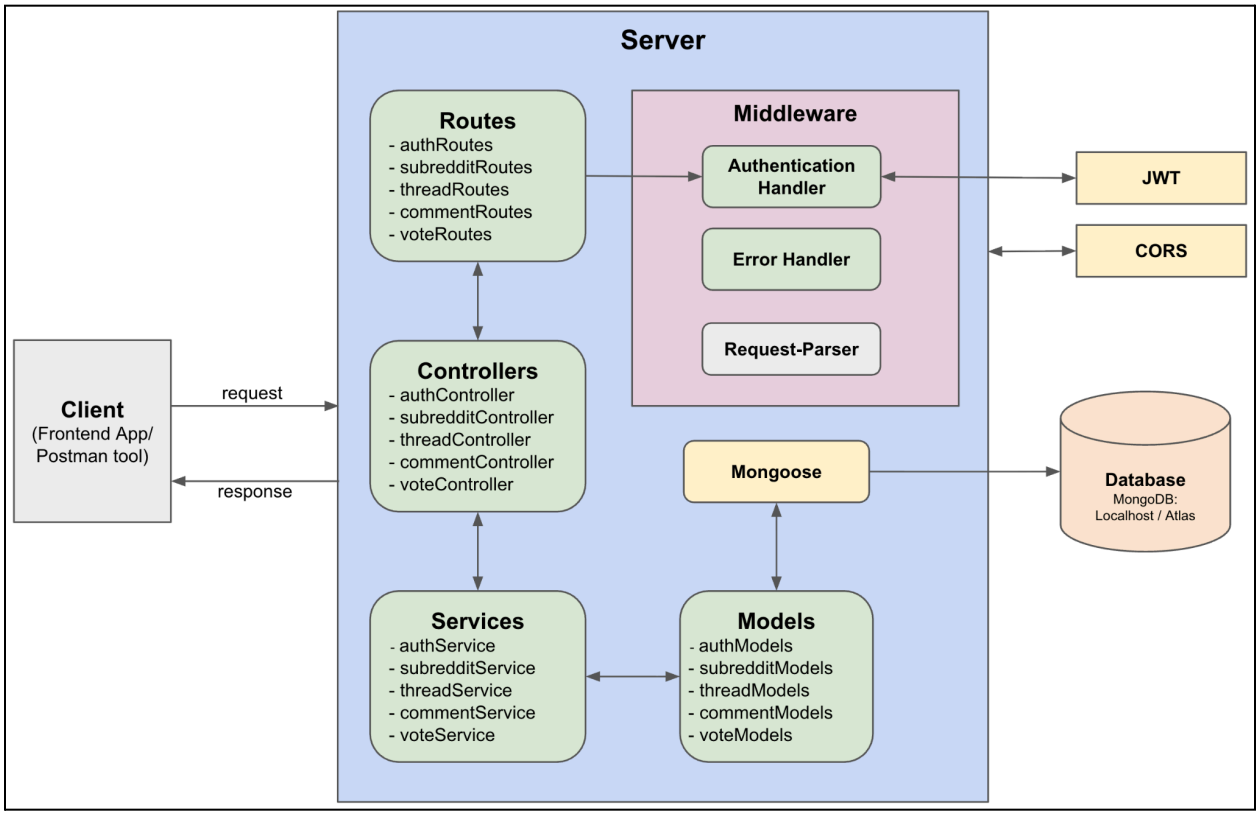
ThreadHive - Design Doc

Introduction

This document details the design and architecture for the backend of the **ThreadHive** application, which is built using **ExpressJS**. It covers the system's setup, API endpoints, data models, and the implemented security and error handling mechanisms.

Backend Server Architecture

The server employs a layered architecture to ensure a clear separation of concerns, with data flowing from client requests through various components to the database and back.



Architecture Details

Component	Role	Key Functionality
Client	External entity	Initiates requests and receives responses.
Server Application	Main logic container	Encapsulates all server-side logic.

Component	Role	Key Functionality
Middleware	Intercepts requests	**Authentication Handler:** Verifies JWT tokens. **Error Handler:** Manages global error response. **Request Parser:** Inbuilt body parser in Express to parse request before they reach route handlers
Routes	API interface	Defines the public endpoints exposed by the server.
Controllers	Business logic	Processes data received from routes and prepares it for services.
Services	Data Access Layer (DAL)	Implements database CRUD operations (Create, Read, Update, Delete).
Models	Data structure definition	Defines the schema for data storage (using Mongoose).
Database	Data persistence	MongoDB instance (local or cloud-hosted) for storing application data.
JWT	External library	Generates and validates JSON Web Tokens for secure authentication.
CORS	External library	Handles Cross-Origin Resource Sharing for secure client-server communication.
Mongoose	ODM	Facilitates structured interaction and data modeling with MongoDB.

Setup and Configuration

Prerequisites

- Node.js must be installed.
- MongoDB database instance is required.
- Environment variables must be configured (e.g., Database URI, Secret Keys).

Installation

1. Clone the repository.
2. Run npm install to download dependencies.
3. Start the server with npm start.
4. Access the server at http://localhost:3000

User Flow and Access Control

User interaction begins with registration/login, after which authenticated users gain full access to the application's core features.

1. **Registration:** Create a user account.
2. **Login:** Authenticate to receive a session token (JWT).
3. **Authenticated Access:** Once logged in, users can perform the following:
    - **Subreddits:** Browse all, view a specific subreddit (with threads), and create new ones.
    - **Threads:** Browse all, view/create/update/delete specific threads.
    - **Comments:** View comments for a thread and add new comments.
    - **Votes:** Upvote or downvote any thread or comment.
4. **Security Policy:** All routes, except /register and /login, require authentication. The request must include the header Authorization: Bearer <token>.
5. **Role Management:** There are no distinct admin roles; all authenticated users possess the same level of access.
6. **Unauthenticated Access:** Only register and login operations can be performed when a user is logged out (token discarded).

Sample Flow

1. **Register User** (POST: http://localhost:3000/api/auth/register) with body:

```JSON
{
   "name": "John Doe",
   "email": "johnd@gmail.com",
   "password": "johnd123"
}
```

2. **Login User** (POST: http://localhost:3000/api/auth/login) with body:

```JSON
{
   "email": "johnd@gmail.com",
   "password": "johnd123"
}
```

3. **Token Usage:** Copy the returned JWT token and use it in the Authorization header (Bearer <token>) for all subsequent requests.
4. **Operations:** Use the API endpoints listed below to interact with the application.
5. **Logout:** Discard the token to log out.

API Endpoints

All endpoints are prefixed with **/api**

Auth/User Routes

Method	Endpoint	Description	Authentication
POST	/auth/register	Creates a new user account.	No
POST	/auth/login	Authenticates user and returns a JWT token.	No

Subreddit Routes

Method	Endpoint	Description	Authentication
GET	/subreddits	Lists all subreddits.	Yes
POST	/subreddits	Creates a new subreddit.	Yes
GET	/subreddits/:id	Retrieves subreddit details including its threads.	Yes

Thread Routes

Method	Endpoint	Description	Authentication
GET	/threads	Lists all threads.	Yes
GET	/threads/:id	Retrieves a single thread.	Yes
POST	/threads	Creates a new thread.	Yes
PUT	/threads/:id	Updates an existing thread.	Yes
DELETE	/threads/:id	Deletes a thread.	Yes

Comment Routes

Method	Endpoint	Description	Authentication
GET	/comments/thread/:threadId	Lists comments for a specified thread.	Yes
POST	/comments	Adds a new comment to a thread.	Yes

Vote Routes

Method	Endpoint	Description	Authentication
POST	/threads/:id/upvote	Upvotes a thread.	Yes

Method	Endpoint	Description	Authentication
POST	/threads/:id/downvote	Downvotes a thread.	Yes
POST	/comments/:id/upvote	Upvotes a comment.	Yes
POST	/comments/:id/downvote	Downvotes a comment.	Yes

Data Models

The following schemas define the structure of data stored in MongoDB.

User

Field	Type	Description
name	String	User's full name.
email	String	Unique email address (used for login).
password	String	Hashed password.

Subreddit

Field	Type	Description
name	String	Name of subreddit
description	String	Main content of subreddit
author	ObjectId	Reference to the user who created the Subreddit.

Thread

Field	Type	Description
title	String	Title of the thread/post.
content	String	Body/main content of the thread.
author	ObjectId	Reference to the user who created the thread.
subreddit	ObjectId	Reference to the subreddit the thread belongs to.
upvotes	Number	Total count of upvotes.
downvotes	Number	Total count of downvotes.
voteCount	Number	Calculated net votes (upvotes - downvotes).
upvotedBy	[ObjectId]	Array of User IDs who have upvoted.
downvotedBy	[ObjectId]	Array of User IDs who have downvoted.

Comment

Field	Type	Description
thread	[ObjectId]	Reference to the thread the comment belongs to.
user	[ObjectId]	Reference to the user who posted the comment.
content	String	Text content of the comment.
upvotedBy	[ObjectId]	Array of User IDs who have upvoted the comment.
downvotedBy	[ObjectId]	Array of User IDs who have downvoted the comment.
voteCount	Number	Calculated net votes.

Security and Authentication

The ThreadHive backend secures all protected endpoints using JWT-based authentication:

- **JWT Usage**
  Upon successful login, the server issues a JSON Web Token (JWT) signed with a server-side secret. This token encodes the user's identity and is returned to the client. The client includes this token in the `Authorization: Bearer <token>` header on subsequent requests.
- **Token Verification Middleware**
  An authentication middleware intercepts incoming requests to protected routes and:
  1. Extracts the JWT from the `Authorization` header.
  2. Verifies the token's signature and expiry time.
  3. Decodes the payload and attaches the authenticated user information (e.g., `req.user`) to the request object.
- If the token is missing, invalid, or expired, the middleware stops the request and returns an appropriate `Unauthorized` response.

Error Handling

A dedicated global error handling middleware is implemented to consistently catch and format responses for errors across the application, ensuring graceful failure and informative feedback to the client.

Conclusion

This design document establishes the foundation for a scalable, secure, and efficient ExpressJS-based backend for the ThreadHive application. The defined architecture, API endpoints, and data models provide a comprehensive blueprint for development and deployment.